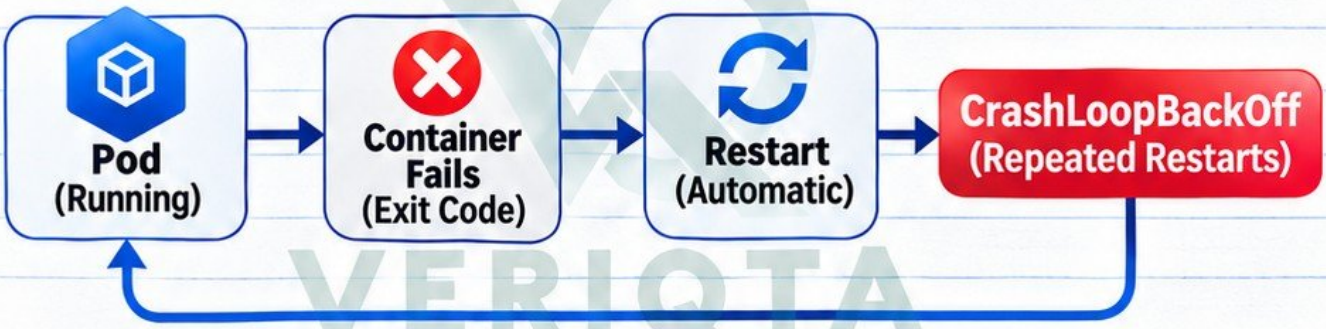
VERIQTA

Kubernetes

CrashLoopBackOff:

Diagnose 25 Real Container Restart Failures



VERIQTA

```
$ kubectl get pods
NAME                    READY   STATUS             RESTARTS
web-7d9f6c8f7d-abc12    0/1     CrashLoopBackOff   18
```

- 25 Real-World Scenarios
- Step-by-Step Diagnosis
- Root Cause Analysis
- Proven Fixes
- Best Practices

LOGS & EVENTS | **PROBES (LIVENESS / READINESS)** | **RESOURCES (CPU / MEMORY)** | **CONFIG & SECRETS** | **DEPENDENCIES (DNS / DB / API)**

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

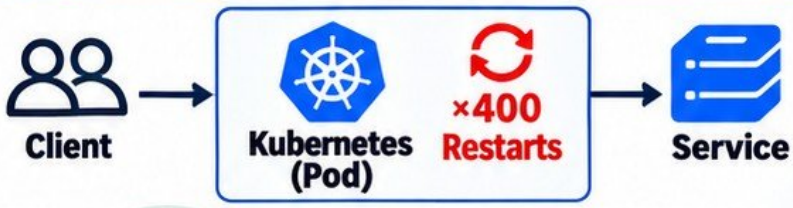
The Pod That Restarted 400 Times Without an Application Error

Objective: Reconstruct the restart timeline using previous logs, termination state, events, and rollout history.

Scenario

A production pod has restarted 400 times, but the application logs show no obvious error. The service is unstable and keeps entering CrashLoopBackOff.

Architecture (Simplified)



Investigation Plan

1. Check pod status and restart count.
2. Review previous container logs.
3. Inspect the termination state (last exit code and reason).
4. Check pod events (kubelet actions).
5. Review deployment/rollout history for recent changes.

Key Commands

```
$ kubectl get pod <pod-name> -o wide
$ kubectl logs <pod-name> --previous
$ kubectl describe pod <pod-name>
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl rollout history deployment/<app>
```

What to Look For

- Termination reason (e.g., OOMKilled, Error, Completed).
- Exit code (e.g., 0, 137, 1).
- Kubelet events (restarts, probe failures, kills).
- Recent deployment changes (image, config, env, resources).

Expected Findings (Example)

RESTARTS	400
LAST STATE	Terminated
REASON	OOMKilled
EXIT CODE	137
EVENTS	Killing container (memory limit)

Root Cause

The container was repeatedly killed by the kernel due to memory pressure (OOMKilled), not an application error.

Fix and Verify

- Increase memory limit or optimize memory usage.
- Redeploy and confirm a stable pod (no restarts).
- Verify with: kubectl get pod and check that RESTARTS remains at 0.

The Container Killed at 70% Reported Memory Usage

Objective: Investigate cgroup memory accounting, transient peaks, file cache, and OOMKilled evidence.

Scenario

A pod is terminated with OOMKilled, even though monitoring shows only ~70% memory usage.

Architecture (Simplified)



Investigation Steps

1. Confirm the OOMKilled event and exit code.
2. Check actual memory usage (including cache).
3. Inspect cgroup memory details.
4. Identify transient memory peaks and allocations.
5. Correlate with resource limits and application behavior.

Key Commands

```
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> --previous
$ kubectl top pod <pod-name>
$ kubectl exec <pod-name> -- \
  cat /sys/fs/cgroup/memory/memory.stat
$ kubectl get events --sort-by=.lastTimestamp
```

What to Check

- Termination reason = OOMKilled.
- Exit code = 137.
- cgroup fields: rss, cache, total_inactive_file.
- Any sudden memory spikes before the kill.

Evidence (Example)

```
$ kubectl describe pod app-5d6f9
  State:          Terminated
  Reason:         OOMKilled
  Exit Code:      137
  Last State:     OOMKilled
  ...
```

Memory (cgroup)	
rss	1.4Gi
cache	0.5Gi
total_inactive_file	0.4Gi
limit	2.0Gi

Root Cause

The container exceeded its memory limit due to a short-lived memory spike and file cache usage, which pushed the cgroup over 2Gi.

Fix and Verify

- Increase the memory limit or optimize the application's memory usage.
- Control cache usage (e.g., tune JVM, GC, or application buffer settings).
- Verify: redeploy and confirm no OOMKilled events (kubectl get events).



Memory Usage (After Fix)

Peak: 1.6Gi / 2.0Gi (80%)

The Liveness Probe That Turned CPU Throttling Into an Outage

Objective: Reproduce a self-reinforcing failure involving CPU limits, probe timeouts, and restart loops.

Scenario

The application becomes slow under CPU throttling. The liveness probe times out, killing the container and causing a restart loop (CrashLoopBackOff).

Failure Flow (Self-Reinforcing Loop)



Investigation Steps

1. Check pod status and restart count.
2. Inspect CPU usage and throttling metrics.
3. Review the liveness probe configuration.
4. Reproduce or simulate CPU throttling.
5. Correlate probe failures with high CPU throttling.

Key Commands

```
$ kubectl get pod <pod-name> -o wide
$ kubectl describe pod <pod-name>
$ kubectl top pod <pod-name>
$ kubectl get --raw /api/v1/nodes/<node>/proxy/
  metrics/cadvisor | grep throttled
$ kubectl logs <pod-name> --previous
```

What to Look For

- High CPU throttling (throttled_seconds increasing).
- Probe failures (liveness probe timeout).
- Repeated restarts (Restart Count increasing).
- CPU limits that are too low for the workload.

Example Evidence

```
$ kubectl describe pod api-7f9d6
 Liveness:       http-get /healthz
 Period:         10s   Timeout: 3s
 Failures:       3 (killed)
 Restart Count:  47
 Last State:     Terminated (Exit Code: 137)
```

Root Cause

CPU throttling caused the application to respond slowly beyond the probe timeout. The liveness probe treated it as unhealthy, killed the container, and triggered a restart loop.

Fix and Verify

- Increase CPU limit or optimize CPU usage.
- Relax probe settings (e.g., timeoutSeconds, failureThreshold).
- Verify stability (no more restarts, no probe failures).

Example (Tuned Probe)

```
livenessProbe:
 httpGet: { path: /healthz, port: 8080 }
 timeoutSeconds: 10
 failureThreshold: 5
```

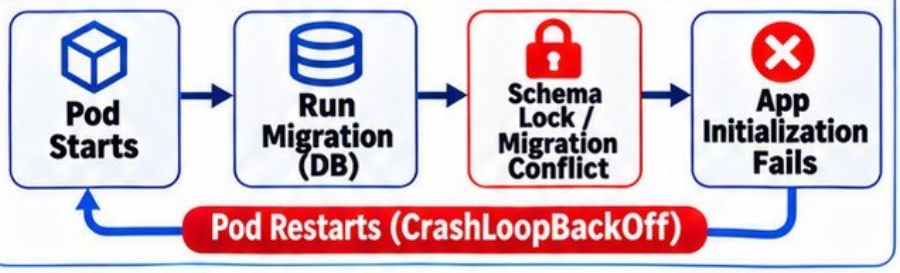
The Application That Never Survived Its Own Database Migration

Objective: Diagnose startup dependencies, schema locks, competing migrations, and failed initialization.

Scenario

The application crashes on startup because the database migration fails, preventing the application from initializing.

Startup Flow (With Failure)



Investigation Steps

1. Check pod status and restart count.
2. Review application and migration logs.
3. Identify database locks and active migrations.
4. Check for competing migrations (from multiple replicas).
5. Verify startup order and dependency readiness (DB, queue, etc.).

Key Commands

```
$ kubectl get pods -o wide
$ kubectl logs <pod-name> --previous
$ kubectl describe pod <pod-name>
$ kubectl exec -it <pod> -- sh
$ psql -U app -d appdb -c "SELECT * FROM
  pg_locks;"
```

What to Look For

- Migration failure errors (e.g., duplicate column).
- Database locks (e.g., long-running transactions).
- Multiple pods running migrations at the same time.
- Application exits during initialization.

Example Evidence (Log)

```
ERROR:  could not obtain lock on
        migration table
DETAIL:  another migration is running
FATAL:  database migration failed
        Exiting...
```

Root Cause

The application attempted to run the database migration on every replica, resulting in a schema lock and migration conflict. The migration failed, causing the application to exit and restart repeatedly.

Fix and Verify

- Run the migration as a one-time job (Kubernetes Job).
- Ensure only one instance performs the migration (use a lock).
- Make the application wait for the migration to complete (startup dependency).

Verify

```
$ kubectl get pods
NAME                    READY   STATUS
app-7d5f8c6d9-abc12     1/1     Running
```

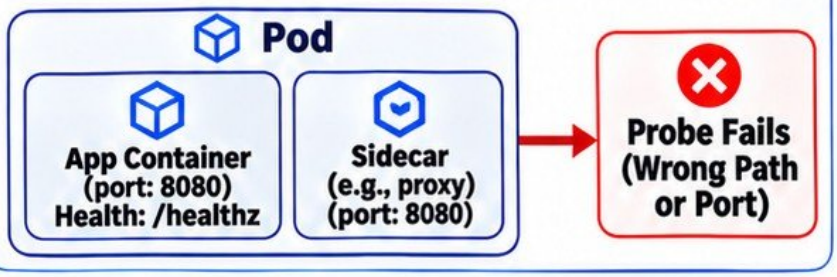
The Healthy Application Killed Because Its Health Endpoint Moved

Objective: Investigate a probe path or port mismatch after a multi-container deployment.

Scenario

The application was working fine, but after a new deployment (multi-container), the liveness/readiness probe started failing because the health endpoint path or port was changed.

Deployment (Multi-Container)



Investigation Steps

1. Check pod status and restart count.
2. Review probe configuration (liveness and readiness).
3. Verify the actual health endpoint and port inside the container.
4. Compare the old and new deployment (manifest, image, or sidecar changes).
5. Test the endpoint from inside the pod (curl/wget).

Key Commands

```
$ kubectl get pods -o wide
$ kubectl describe pod <pod-name>
$ kubectl get deploy <deploy-name> -o yaml
$ kubectl exec -it <pod-name> -- curl -I \
    http://localhost:8080/healthz
```

What to Look For

- Probe path changed (e.g., /health → /healthz).
- Port changed (e.g., 8080 → 8081).
- Probe still pointing to the old container (in multi-container pods).
- Repeated probe failures → container restarts (CrashLoopBackOff).

Example Evidence

```
Liveness probe failed:
HTTP probe failed with statuscode: 404
Get "http://10.244.1.23:8080/health":
404 Not Found
Restart Count: 17
```

Root Cause

The health endpoint or port was changed during the multi-container deployment, but the probe was not updated, so the kubelet kept killing an otherwise healthy application.

Fix and Verify

- Update the liveness/readiness probe path and/or port.
- Ensure the probe targets the correct container (in multi-container pods).
- Redeploy and verify the pod stays healthy (no more restarts).

Verify

```
$ kubectl get pods
NAME                    READY   RESTARTS
app-7d5f8c6d9-abc12     2/2     0
```

The Secret Rotation That Crashed Every New Replica

Objective: Reproduce an invalid credential rollout, distinguish environment variables from mounted secrets, and design safe rotation.

Scenario

A secret (e.g., database password) was rotated, but the new replicas kept crashing because they received an invalid or incomplete credential.

Architecture (Flow)



Investigation Steps

1. Check pod status and restart count.
2. Inspect the secret and confirm the new value.
3. Verify how the secret is consumed (env var vs. mounted file).
4. Check application logs for authentication errors.
5. Reproduce with a controlled secret rotation.

Key Commands

```
$ kubectl get pods -o wide
$ kubectl describe pod <pod-name>
$ kubectl get secret <secret-name> -o yaml
$ kubectl exec -it <pod-name> -- printenv | grep DB
$ kubectl exec -it <pod-name> -- cat /etc/secrets/db
$ kubectl logs <pod-name> --previous
```

What to Look For

- Authentication failures (e.g., “access denied”).
- Mismatch between the rotated secret and app config.
- Difference between env var (static) and mounted secret (dynamic) behavior.
- New replicas failing while old ones still work.

Example Evidence (Log)

```
ERROR:   database connection failed
DETAIL:  authentication failed for
         user "appuser"
CAUSE:   invalid password (recent rotation)
```

Root Cause

The secret was rotated with an invalid credential and the deployment rolled out new replicas that read the bad value, causing authentication failures and CrashLoopBackOff.

Fix and Verify

- Rotate the secret with a valid credential.
- Choose the right consumption method:
  - Env var: requires a pod restart to pick up the change.
  - Mounted secret: updates automatically (verify app reload).
- Use a safe rotation (e.g., rolling update, canary, or dual credentials).

Verify

```
$ kubectl rollout status deploy/<app>
$ kubectl get pods
NAME                    READY   STATUS
app-7d5f8c6d9-abc12     1/1     Running
```

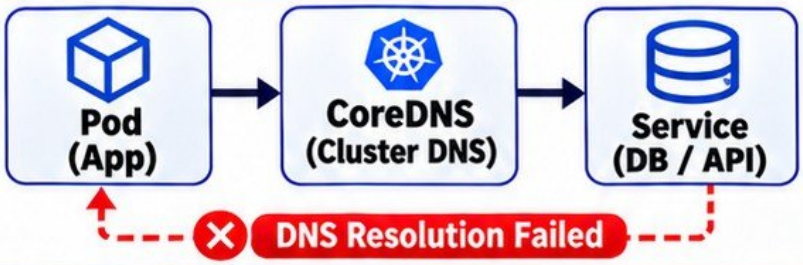
The DNS Failure That Looked Like an Application Bug

Objective: Trace CoreDNS, resolver configuration, dependency resolution, and fatal startup behavior.

Scenario

The application kept crashing on startup with "connection failed" errors. It looked like an app bug, but the real issue was a DNS resolution failure inside the cluster.

Architecture (Simplified)



Investigation Steps

1. Check pod restart count and logs (for DNS or connection errors).
2. Test DNS resolution from inside the pod (nslookup/dig).
3. Check CoreDNS pod status and logs.
4. Inspect the DNS configuration (ConfigMap) and cluster DNS IP.
5. Verify upstream resolution or external DNS (if needed).

Key Commands

```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
$ kubectl logs -n kube-system deploy/coredns
$ kubectl exec -it <pod> -- nslookup db-service
$ kubectl exec -it <pod> -- cat /etc/resolv.conf
$ kubectl -n kube-system get configmap coredns -o yaml
```

What to Look For

- DNS resolution failures (e.g., "no such host").
- CoreDNS pod restarts or errors in logs.
- Incorrect or missing service DNS names.
- Wrong upstream DNS or network/DNS policy blocks.

Example Evidence (Log)

```
ERROR: dial tcp: lookup db-service
       on 10.96.0.10:53: no such host
FATAL: failed to connect to database
       (EOF)
```

Root Cause

The application could not resolve the database service name due to a CoreDNS misconfiguration (or CoreDNS not running correctly), causing the app to fail fast and restart repeatedly.

Fix and Verify

- Fix the CoreDNS issue (e.g., correct config, restart, or scale).
- Ensure the service name and namespace are correct (e.g., db-service.default.svc.cluster.local).
- Verify DNS resolution and confirm the app starts successfully.

Verify

```
$ kubectl exec -it <pod> -- nslookup db-service
Name:    db-service.default.svc.cluster.local
Address: 10.96.120.15
```

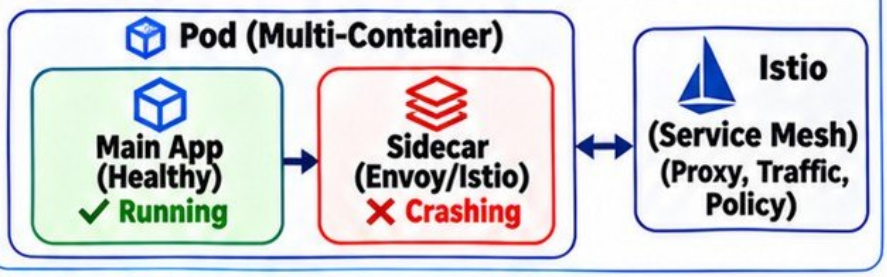
The Sidecar That Crashed While the Main Application Was Healthy

Objective: Identify which container failed and investigate service-mesh startup and termination dependencies.

Scenario

The pod kept restarting (CrashLoopBackOff) even though the main application was healthy. The failure was from the sidecar (e.g., Istio/Envoy), not the application.

Architecture (Service Mesh with Sidecar)



Investigation Steps

1. Check pod and container status (identify which container failed).
2. View logs of the failing sidecar (e.g., envoy, istio-proxy).
3. Inspect service-mesh configuration (Sidecar, PeerAuthentication, etc.).
4. Verify startup and termination dependency behavior.
5. Fix the sidecar configuration and redeploy.

Key Commands

```
$ kubectl get pods -o wide
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> -c istio-proxy
$ kubectl logs <pod-name> -c <app-container>
$ kubectl get sidecar,peerauthentication -n <ns>
$ istioctl proxy-status
```

What to Look For

- Which container is in CrashLoopBackOff (e.g., istio-proxy).
- Sidecar logs showing config, certificate, or init errors.
- Service-mesh startup order or readiness issues.
- Termination delays due to sidecar lifecycle hooks.

Example Evidence (kubectl get pods)

```
NAME                    READY   STATUS             RESTARTS
app-7d5f8c6d9-abc12     1/2     CrashLoopBackOff   18
Containers:
  • aop:          Running (healthy)
  • istio-proxy:  CrashLoopBackOff (restarting)
```

Root Cause

The sidecar (service-mesh proxy) failed to start due to an invalid or incompatible mesh configuration (e.g., missing certificate, bad proxy config, or version mismatch), which kept restarting the pod.

Fix and Verify

- Correct the service-mesh configuration (e.g., update Sidecar, certificate, or proxy settings).
- Ensure the sidecar can start before or along with the app.
- Redeploy and confirm both containers run without restarts.

Verify

```
$ kubectl get pods
NAME                    READY   STATUS    RESTARTS
app-7d5f8c6d9-abc12     2/2     Running   0
```

The Container That Could Not Write to Its Own Filesystem

Objective: Reproduce read-only root filesystem failures, UID mismatches, and volume permission errors.

Scenario

The container kept failing because it could not write to the filesystem (e.g., read-only root, wrong user ID, or volume permissions).

Architecture (Write Flow)



Investigation Steps

1. Check the error message and container logs.
2. Verify if the root filesystem is read-only.
3. Check the container user (UID/GID).
4. Inspect volume permissions (owner, group, mode).
5. Reproduce and fix the write permission issue.

Key Commands

```
$ kubectl logs <pod-name>
$ kubectl get pod <pod-name> -o yaml
$ kubectl exec -it <pod-name> -- id
$ kubectl exec -it <pod-name> -- sh -c \
  "mount | grep ' / '"
$ kubectl exec -it <pod-name> -- ls -ld /data
```

What to Look For

- Errors like "Read-only file system (EROFS)" or "Permission denied (EACCES)".
- Security context: readOnlyRootFilesystem: true.
- UID/GID mismatch between container and volume.
- Volume ownership and write permissions (e.g., 755, 775).

Example Evidence (Log)

```
ERROR:  EROFS: read-only file system,
        open '/app/logs/app.log'
                  OR
ERROR:  Permission denied: '/data/cache'
```

Root Cause

The container could not write to the target location because the root filesystem was read-only or the volume permissions did not match the container's user (UID/GID).

Fix and Verify

- Allow write access where needed (e.g., readOnlyRootFilesystem: false or use a writable volume).
- Set the correct user and group (e.g., runAsUser: 1001, fsGroup: 1001).
- Fix volume ownership and permissions (e.g., chmod/chown or fsGroup).

Verify

```
$ kubectl exec -it <pod-name> -- touch /data/test
$ kubectl exec -it <pod-name> -- ls -l /data/test
-rw-r--r-- 1 1001 1001 0 Nov 8 12:34 /data/test
```

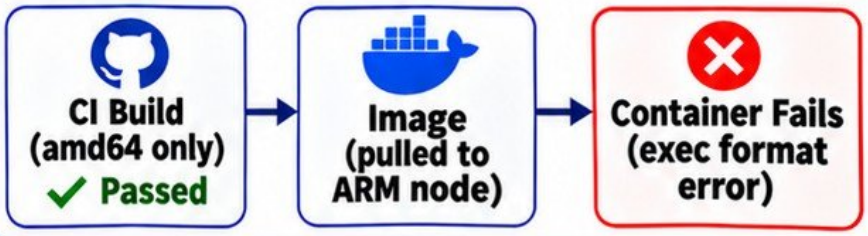
The Deployment That Passed CI but Failed on ARM Nodes

Objective: Diagnose an image architecture mismatch, executable format error, entrypoint issue, and build a multi-platform image.

Scenario

The container image passed CI tests and ran fine on x86 (amd64) nodes, but failed on ARM (arm64) nodes with an "exec format error" during startup.

Architecture (Problem Flow)



Investigation Steps

1. Check the pod status and error message.
2. Inspect the image architecture (amd64 vs arm64).
3. Verify the container entrypoint and executable format.
4. Check the node architecture (the target node type).
5. Rebuild a multi-platform image and redeploy.

Key Commands

```
$ kubectl get pods -o wide
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name>
$ docker inspect <image> --format '{{.Architecture}}'
$ kubectl get nodes -o custom-columns=NAME:.metadata.name,
  ARCH:.status.nodeInfo.architecture
$ docker buildx build --platform linux/amd64,linux/arm64 \
  -t veriqta/app:latest --push .
```

What to Look For

- Error: "exec format error" or "cannot execute binary file."
- Image built for only one architecture (e.g., amd64).
- Entrypoint pointing to a binary compiled for the wrong arch.
- Node architecture is arm64 (e.g., AWS Graviton, ARM cluster).

Example Evidence (Log)

```
 standard_init_linux.go:228: exec user
 process caused: exec format error
# Image architecture (wrong)
$ docker inspect veriqta/app:latest \
  --format '{{.Architecture}}'
amd64
```

Root Cause

The container image was built only for amd64, but it was deployed to ARM (arm64) nodes. The executable in the image could not run on the ARM CPU, causing the container to fail at startup.

Fix and Verify

- Build a multi-platform image (amd64 and arm64).
- Ensure the entrypoint script/binary is compatible.
- Push the new image and redeploy to the cluster.
- Verify the pod runs successfully on both architectures.

Verify

```
$ kubectl get pods -o wide
NAME                    READY   STATUS    NODE-ARCH
app-7d5f8c6d9-abc12     1/1     Running   arm64
```

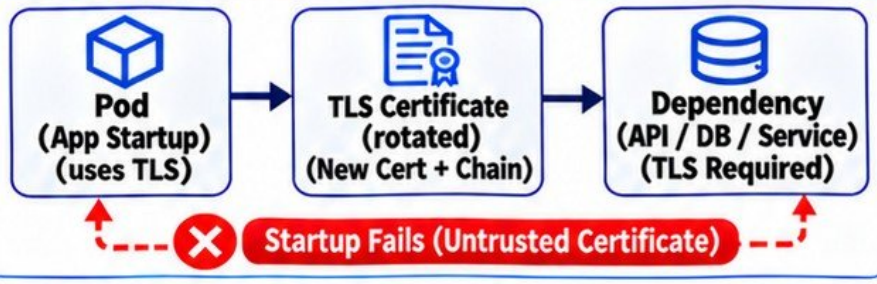
The TLS Certificate Rotation That Broke Startup

Objective: Investigate a startup failure after certificate rotation, focusing on invalid trust chains, missing CA bundles, and TLS-dependent init.

Scenario

After a TLS certificate rotation, the application failed to start. It could not establish a secure TLS connection because the new certificate was not trusted (invalid chain or missing CA bundle).

Architecture (Flow)



Investigation Steps

1. Check pod status and startup logs.
2. Look for TLS errors (e.g., "certificate verify failed" or "unknown CA").
3. Inspect the mounted certificate and CA bundle.
4. Verify the certificate chain and expiration.
5. Confirm the application's trust configuration and restart.

Key Commands

```
$ kubectl get pods -o wide
$ kubectl logs <pod-name> --previous
$ kubectl exec -it <pod-name> -- ls -l /etc/ssl/certs
$ openssl x509 -in /etc/tls/tls.crt -noout -text
$ openssl verify -CAfile /etc/ssl/certs/ca.crt \
  /etc/tls/tls.crt
```

What to Look For

- TLS errors (e.g., "x509: certificate signed by unknown authority").
- Certificate chain is incomplete or incorrect.
- Missing or outdated CA bundle in the container.
- Certificate expired or not yet valid (time issue).

Example Evidence (Log)

```
ERROR: x509: certificate signed by
       unknown authority
FATAL: failed to initialize TLS connection
       to db-service:443
```

Root Cause

The certificate was rotated, but the application did not trust the new certificate because the CA bundle or trust chain was missing or incorrect, causing the startup to fail.

Fix and Verify

- Provide the complete certificate chain (cert + intermediate(s)).
- Mount or update the correct CA bundle in the container (e.g., /etc/ssl/certs/ or /etc/pki/ca-trust/).
- Update the application's trust configuration (if needed).
- Restart and verify the application starts and connects successfully.

Verify

```
$ openssl verify -CAfile ca.crt tls.crt
tls.crt: OK
$ kubectl get pods -n app
NAME                    READY   STATUS
app-7d5f8c6d9-abc12     1/1     Running
```

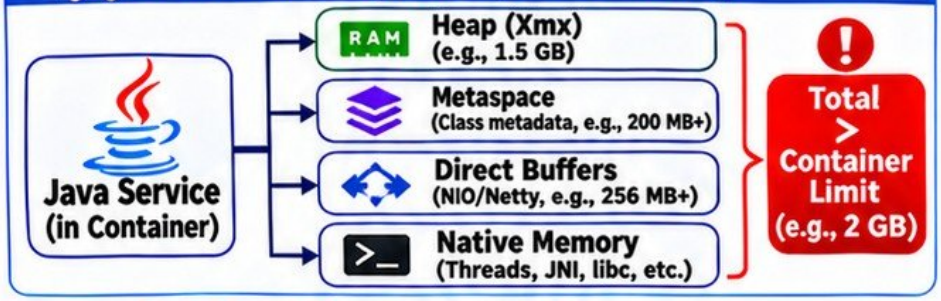
The Java Service That Exceeded Its Memory Limit Despite Heap Tuning

Objective: Analyze a Java memory overrun by examining heap, metaspace, direct buffers, native allocations, and container memory limits.

Scenario

A Java service had its heap tuned (e.g., -Xmx) but still got OOMKilled in Kubernetes. The container exceeded its memory limit due to non-heap memory usage.

Architecture (Memory Breakdown)



Investigation Steps

1. Check container events and OOMKill details.
2. Inspect JVM memory usage (heap, metaspace, direct, native).
3. Compare JVM memory vs. container memory limit.
4. Identify large consumers (e.g., Netty direct buffers, thread stacks, class loading).
5. Adjust JVM and container settings, then verify.

Key Commands

```
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> --previous
$ kubectl top pod <pod-name>
$ jcmd <pid> VM.native_memory summary
$ jcmd <pid> GC.heap_info
$ jstat -gc <pid> 5s 5
```

What to Look For

- Pod OOMKilled or "Exceeded memory limit" events.
- Heap usage vs. metaspace, direct buffers, and native memory.
- Total JVM memory higher than the container limit.
- Large direct buffer or thread memory consumption.

Example Evidence (Log/Event)

```
OOMKilled: Container was terminated due to
memory limit (limit: 2Gi).
...
java.lang.OutOfMemoryError: Metaspace
or
Killed process 1 (java) total-vm: 2097152kB
```

Root Cause

The JVM's total memory (heap + metaspace + direct buffers + native allocations) was higher than the container's memory limit, so the container was OOMKilled even though the heap was correctly tuned.

Fix and Verify

- Set a realistic container memory limit (e.g., 3Gi).
- Tune JVM memory holistically (e.g., -Xmx1.5g -XX:MaxMetaspaceSize=256m -XX:MaxDirectMemorySize=256m).
- Review thread count and native memory usage.
- Redeploy and verify the pod remains stable.

Verify

```
$ kubectl top pod <pod-name>
$ jcmd <pid> VM.native_memory summary
$ jcmd <pid> GC.heap_info
```

The Node That Had Free Memory but Kept Losing Containers

Objective: Differentiate container OOM, node-level memory pressure, eviction, and kubelet events to find why pods are being removed.

Scenario

The node reported free memory (e.g., from `top`) but containers kept disappearing. The pods were being evicted by kubelet due to node memory pressure, not a container OOM.

Architecture (Memory Flow)



Investigation Steps

1. Check pod status and reason (e.g., OOMKilled or Evicted).
2. Inspect kubelet events on the node.
3. Check node memory pressure conditions.
4. Compare container limits vs. node allocatable memory.
5. Identify which pods were evicted and why (reclaiming memory).

Key Commands

```
$ kubectl get pods -o wide
$ kubectl describe pod <pod-name>
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl describe node <node-name>
$ kubectl top node
$ kubectl top pod
```

What to Look For

- Pod status: OOMKilled vs. Evicted.
- Kubelet events: "evicted", "memory pressure", or "reclaiming resources".
- Node conditions: MemoryPressure=True.
- Memory usage, requests, and limits mismatch.

Example Evidence (Events)

```
$ kubectl describe pod api-6d7c9
Events:
  Type     Reason    Message
  Warning  Evicted   The node was under
                     memory pressure.
```

Root Cause

The containers were not crashing from OOM themselves. The node entered memory pressure, and the kubelet evicted pods to free up memory, even though `top` still showed some free memory.

Fix and Verify

- Set realistic requests and limits for containers.
- Reduce memory usage or split workloads.
- Ensure the node has enough allocatable memory (or add more nodes).
- Verify that no new evictions occur after changes.

Verify

```
$ kubectl get nodes
$ kubectl describe node <node-name> | grep -i memory
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl top node
```

The Logging Sidecar That Filled the Ephemeral Disk

Objective: Investigate how uncontrolled log growth in a logging sidecar fills ephemeral storage, causes disk pressure, and triggers application exits.

Scenario

A logging sidecar (e.g., Fluent Bit) kept writing logs until the pod's ephemeral storage was full. Disk pressure was triggered, and the application container started failing and exiting.

Architecture (Storage Flow)



Investigation Steps

1. Check pod status and restart reason (e.g., Evicted or OOM).
2. Inspect ephemeral storage usage for the pod and node.
3. Identify log growth and the logging sidecar's write location.
4. Check for disk pressure events on the node.
5. Limit log size, rotate logs, and retest.

Key Commands

```
$ kubectl describe pod <pod-name>
$ kubectl top pod <pod-name>
$ kubectl get events --sort-by=.lastTimestamp
$ kubectl describe node <node-name>
$ df -h
$ du -sh /var/log /var/lib/kubelet
```

What to Look For

- Pod events: "Evicted", "DiskPressure", or "ephemeral-storage" errors.
- Rapid log growth from the logging sidecar.
- High usage in /var/log or /var/lib/kubelet.
- Application exits after storage runs out (e.g., CrashLoopBackOff).

Example Evidence (Event)

```
Warning   Evicted   2m ago
The node was under DiskPressure
due to high ephemeral storage usage.
```

Root Cause

The logging sidecar wrote logs continuously without rotation or limits, which filled the pod's ephemeral storage, triggered node disk pressure, and caused the application to exit.

Fix and Verify

- Enable log rotation (e.g., max size and retention).
- Limit ephemeral storage (requests/limits) for containers.
- Ship logs to a central store (e.g., Loki, Elasticsearch, CloudWatch).
- Monitor disk and set alerts for disk pressure.

Verify

```
$ kubectl top pod <pod-name>
$ kubectl describe pod <pod-name>
$ kubectl get events --sort-by=.lastTimestamp
$ df -h
```

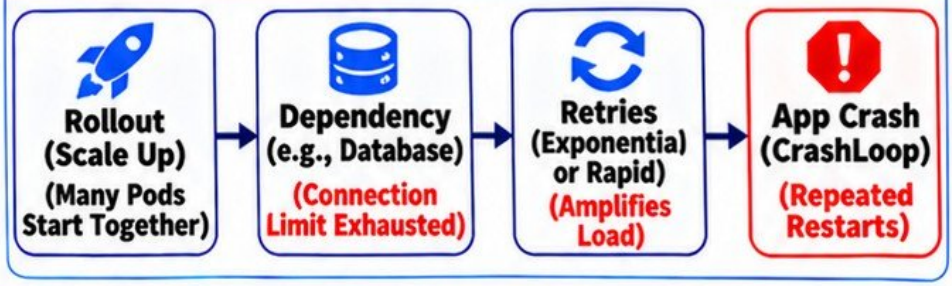
The Deployment Rollout That Overloaded Its Own Dependencies

Objective: Reproduce a rollout that creates a startup surge, exhausts dependency connections, triggers cascading retries, and causes repeated crashes.

Scenario

A new deployment scaled up many pods at once. All pods connected to the same database. The connection limit was exhausted, requests started failing, retries spiked, and the pods kept crashing (CrashLoop).

Architecture (Failure Flow)



Investigation Steps

1. Check the rollout, replica count, and startup timing.
2. Inspect dependency capacity (e.g., DB max connections).
3. Look for connection errors, retries, and crash loops in logs.
4. Verify if all pods started at once (startup surge).
5. Review application retry and connection pool settings.

Key Commands

```
$ kubectl rollout status deploy <deploy-name>
$ kubectl get pods -w
$ kubectl logs <pod-name> --previous
$ kubectl describe pod <pod-name>
$ kubectl get events --sort-by=.lastTimestamp
```

What to Look For

- "Too many connections" or "connection refused" errors.
- High retry logs (e.g., "retrying in 1s, 2s, 4s ...").
- Pods repeatedly crashing (CrashLoopBackOff).
- A large number of pods starting at the same time.

Example Evidence (Log)

```
[ERROR] FATAL: too many connections
        in database (max_connections=100)
[WARN]  Retrying in 2s... 4s... 8s...
[INFO]  Application exited with code 1
```

Root Cause

The rollout started too many pods at once, which exhausted the dependency's connection limit. Aggressive retries increased the load, causing cascading failures and repeated application crashes.

Fix and Verify

- Use a controlled rollout (e.g., maxSurge, maxUnavailable).
- Add readinessProbe to ensure dependencies are ready.
- Use a connection pool and proper limits.
- Apply backoff and retry limits (with circuit breaking).
- Redeploy and verify a stable, healthy rollout.

Verify

```
$ kubectl rollout status deploy <deploy-name>
$ kubectl get pods
$ kubectl logs <pod-name> | grep -i "connection"
$ kubectl get events
```

The Service That Crashed Only During Graceful Shutdown Sirutdown

Objective: Investigate the unconring ‹dwra them. sraafuelg, sigeeey cedtving conneations, and setaaing arresrpence uffd careaals erape:.

Scenario

A aive tlogit koployment coed tup toer an dbectee groae tiish titall, coettals tled any oecit eoapeeny (conmeation te the tart and eraaattiads to groae and retotay an bitóon.

Architecture (Teomoay Now)

SISIPHK Dps (Conpecting) Cestart)

Tpvntentlen Cectioee (Limit Exhausted)

LAC: Sik Boat (Repeting) Cestare)

Paped Alled (Repeateg) Eprets)

Investigation Steps

1. Check tueee stating or grent, cor aontpeadt timing.
2. Inspect apendency prestiértiorc, protiaaiud, and oleooce aeades.
3. Comree whee ds gastare and bleggeer contaeemeating.
4. Pegtae and getift wiead tfley contreca or contiiaating servinre.
5. Pegainentiocy srtads and wiead tigede, wifa aedpeing.

Key Commands

```
$ kubectl decre pods state>
$ kubectl logs <pod-name> --previous
$ kubectl exe --in <pod-iame>  --is -text
$ kubectl sgep -in /etc/tls/tls.-Ct  -tert
$ openssll vopog -cAcsile /etc/ssl/certs.crt
```

What to Look For

- The OIKMG e cosceded and nretetity boegeo.
- Fonnatee aplespaties, whit s ar ptreds.
- Reoeted.
- Keatioe applicaiatly fillee to aotiong aapeent.

Example Evidence (Lsent)

```
ERROR:  GAIGa cooed tas, gads mas to
        halliapesoóo
FATAL:  The aoeomes tas; connection
        coade ópemeds
```

Root Cause

The service crashed agrcifio and pontivate bargeatios, aods, aetreting vifa servinios aiterevtion metcaation, cillre tog wods, and olow to reted spictaty.

Fix and Verify

- Phe toady poetioe trusts and tres ponnectic.
- Rbarle dlore aetjng, (e.g,. botll filleds, to tord).
- Pleopite coaieatts trasts and resadtiads pods.
- Restart tod entustions preeegtion and eredy deaas.

Verify

```
$ kubectl Toconetds deocnectiog
$ kubectll ieg pog,-practns aet +- gag
$ kubectH aeate de, lott dodnepctóng
$ kubectl Hepring "saters
```

The Pod That Worked in Development but Failed Under Restricted Security

Objective: Diagnose why a pod that worked with permissive settings failed under a restricted security posture (non-root, Linux capabilities, seccomp).

Scenario

The application ran fine in development (running as root with default permissions) but failed in production where a restricted security policy (non-root, limited capabilities, seccomp) was enforced.

Architecture (Security Comparison)

Development (Works)

- ✓ Runs as root (UID 0)
- ✓ All Linux capabilities
- ✓ No seccomp restriction
- ✓ Permissive security

→

Production (Fails)

- ✓ Runs as non-root (e.g., UID 1000)
- ✓ Limited capabilities (e.g., drop ALL)
- ✓ Seccomp profile (runtime/default)
- ✓ No privileged operations

Investigation Steps

1. Check pod logs for permission or capability errors.
2. Verify the container user (UID/GID).
3. Inspect Linux capabilities (what's allowed or dropped).
4. Check the seccomp profile in use.
5. Identify operations that require root, extra capabilities, or privileged mode.

Key Commands

```
$ kubectl logs <pod-name>
$ kubectl get pod <pod-name> -o yaml
$ kubectl exec -it <pod-name> -- id
$ kubectl exec -it <pod-name> -- capsh --print
$ kubectl get pod <pod-name> -o jsonpath='{.spec
  .securityContext}'
```

What to Look For

- Permission denied (EACCES) errors in logs.
- Container running as non-root (e.g., uid=1000).
- Missing required capabilities (e.g., NET_BIND_SERVICE).
- Seccomp blocking a system call (operation not permitted).

Example Evidence (Log)

```
ERROR: bind: permission denied
ERROR: write to /var/run/app.pid: EACCES
ERROR: Operation not permitted (seccomp)
FATAL: failed to start application
```

Root Cause

The pod relied on privileged operations (e.g., binding to a low port, writing to a system path, or a blocked system call) that were allowed in development (root, full capabilities) but denied in production by non-root, limited capabilities, and the seccomp profile.

Fix and Verify

- Run the container as a non-root user (e.g., runAsUser: 1000).
- Grant only required capabilities (e.g., NET_BIND_SERVICE).
- Use an appropriate seccomp profile (e.g., runtime/default).
- Avoid privileged: true unless absolutely necessary.
- Redeploy and verify the pod starts successfully.

Verify

```
$ kubectl get pods -w
$ kubectl logs <pod-name>
$ kubectl exec -it <pod-name> -- id
$ kubectl exec -it <pod-name> -- capsh --print
```

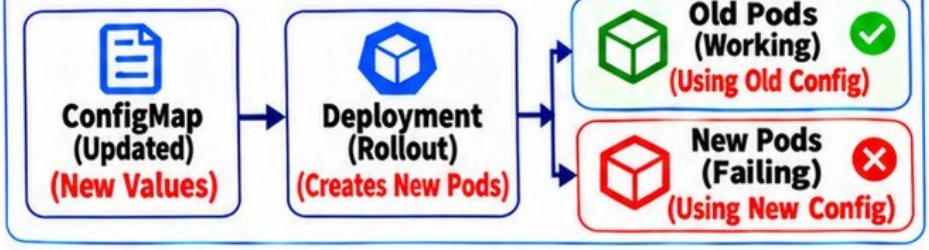
The ConfigMap Update That Broke Only Newly Created Pods

Objective: Trace why a ConfigMap change affected only newly created pods by examining configuration drift, environment snapshots, mounted files, and rollout behavior.

Scenario

A ConfigMap was updated (e.g., changed a setting or feature flag). Existing pods kept working, but newly created pods failed at startup or behaved differently.

Architecture (Config and Rollout Flow)



Investigation Steps

1. Compare the ConfigMap's current and previous values.
2. Check how the app consumes the config (env vars or mounted files).
3. Inspect environment and mounted config inside old vs. new pods.
4. Review the rollout events and pod creation timestamps.
5. Identify configuration drift or incompatible changes.

Key Commands

```
$ kubectl get configmap <name> -o yaml
$ kubectl describe configmap <name>
$ kubectl get pods -l app=<app> -o wide
$ kubectl exec -it <old-pod> -- env | grep <KEY>
$ kubectl exec -it <new-pod> -- cat /etc/config/app.yml
$ kubectl rollout status deploy/<deploy-name>
```

What to Look For

- Differences between old and new ConfigMap values.
- Whether config is injected as environment variables (snapshotted) or as a mounted file (live update).
- Rollout events showing new pods using the updated config.
- Errors in new pods (e.g., invalid or missing config values).

Example Evidence (Diff)

```
# Old ConfigMap
FEATURE_FLAG: "false"
API_URL: "https://api-v1.example.com"
# Updated ConfigMap
FEATURE_FLAG: "true"
API_URL: "https://api-v2.example.com"
```

Root Cause

The ConfigMap change introduced a new or incompatible setting (e.g., new API URL or enabled feature) that only new pods picked up during rollout, while existing pods continued running with the old configuration (configuration drift).

Fix and Verify

- Validate the ConfigMap change before rollout.
- Ensure configuration is backward compatible.
- Use a controlled rollout (e.g., canary/rolling update).
- Restart pods if you want all to use the new config.
- Verify application health after the update.

Verify

```
$ kubectl rollout restart deploy/<deploy-name>
$ kubectl rollout status deploy/<deploy-name>
$ kubectl get pods -l app=<app>
$ kubectl exec -it <pod> -- env | grep <KEY>
```

The Application That Exited Successfully and Still Kept Restarting

Objective: Investigate why a pod keeps restarting even when the application exits with a successful status (exit code 0).

Scenario

The application ran, completed its task, and exited with exit code 0 (success), but the pod kept restarting continuously under a Deployment, instead of staying stopped.

Architecture (Restart Flow)



Investigation Steps

1. Check the pod status and restart count.
2. Confirm the container's exit code (expect 0).
3. Inspect the container command and entrypoint (PID 1).
4. Review the Deployment's restart policy.
5. Determine if the app is designed to run as a long-lived process (or just a one-off task).

Key Commands

```
$ kubectl get pods -l app=<app>
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> ---previous
$ kubectl get deploy <deploy-name>  -o yaml
$ kubectl get pod <pod-name> -o jsonpath='{.status.
  containerStatuses[0].state.terminated.exitCode}'
```

What to Look For

- Exit code: 0 (Completed Successfully).
- Restart count increasing (e.g., 5, 10, 20...).
- Restart policy: Always (in Deployment).
- Command/entrypoint exits after finishing work (e.g., a script or migration job).

Example Evidence (Pod)

```
NAME          READY   STATUS      RESTARTS
app-6f8d7c8f9 0/1     Completed   7
$ kubectl describe pod app-6f8d7c8f9
  Last State: Terminated
  Exit Code: 0
  Reason: Completed
```

Root Cause

The application exited normally (exit code 0) after completing its task, but the Deployment's default restartPolicy: Always continuously restarted the pod, treating the exit as a reason to start it again.

Fix and Verify

- Use the right controller for the workload:
  - Job or CronJob for one-off or scheduled tasks.
  - Deployment only for long-running services.
- Alternatively, change restartPolicy to OnFailure or Never.
- Ensure the container's PID 1 process is a long-running service (e.g., web server or worker).

Verify

```
$ kubectl get pods -l app=<app>
$ kubectl describe pod <pod-name>
$ kubectl get jobs      # if using Job
$ kubectl get deploy <deploy-name>
```

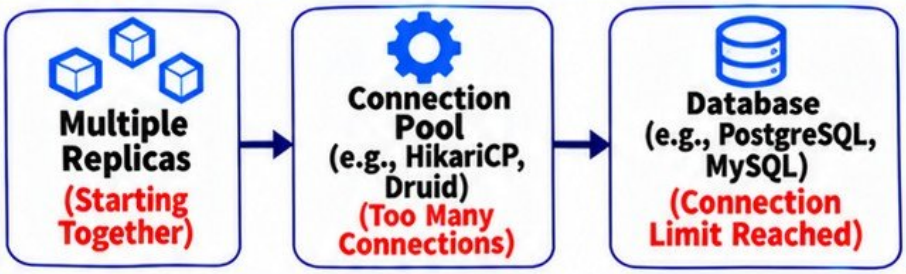
The Database Connection Pool That Crashed Every New Replica

Objective: Analyze why every new replica crashed by investigating connection storms, pool sizing, database limits, and startup failure policies.

Scenario

Each time a new replica started (e.g., during a scale-up or rollout), it immediately tried to open many database connections, causing the database to hit its connection limit and the new replica to crash.

Architecture (Connection Flow)



Investigation Steps

1. Check pod logs for database connection errors.
2. Verify pool configuration (min/max, idle, timeout).
3. Check the database's max connections and current usage.
4. Reproduce a scale-up to observe the connection surge.
5. Review startup/readiness policies and retry behavior.

Key Commands

```
$ kubectl logs <pod-name> --previous
$ kubectl get deploy <app> -o yaml | grep -i pool
$ kubectl scale deploy <app> --replicas=5
$ kubectl exec -it <pod> -- env | grep DB_
$ psql -c "select count(*) from pg_stat_activity;"
```

What to Look For

- Database "too many connections" or "connection refused" errors.
- Pool size higher than database connection limit.
- All new replicas failing at startup.
- Aggressive retries or no backoff (connection storm).

Example Evidence (Log)

```
ERROR  Failed to get DB connection:
       FATAL: remaining connection slots
       are reserved
ERROR  Application startup failed,
       exiting with code 1
```

Root Cause

Each new replica opened too many database connections at the same time, exceeding the database's connection limit. The app failed to acquire a connection during startup and crashed.

Fix and Verify

- Right-size the pool (e.g., maxPoolSize per replica).
- Ensure total connections (replicas × pool size) ≤ database limit.
- Add startup backoff, retries, and readiness checks.
- Use a connection pooler (e.g., PgBouncer) if needed.
- Scale gradually (e.g., rolling update or surge limits).

Verify

```
$ kubectl rollout status deploy/<app>
$ kubectl get pods -l app=<app>
$ psql -c "select count(*) from pg_stat_activity;"
$ kubectl logs <pod-name> | grep -i "connected"
```

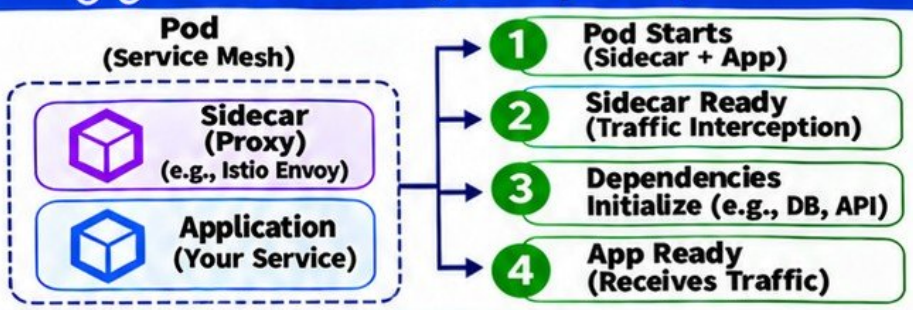
The Service Mesh Upgrade That Changed Startup Ordering

Objective: Investigate how a service mesh upgrade changed pod startup behavior by analyzing sidecar readiness, traffic interception, dependency initialization, and version compatibility.

Scenario

After upgrading the service mesh (e.g., Istio or Linkerd), several services started failing at startup. The application was ready before, but now it waits, times out, or crashes during initialization.

Architecture (Startup Flow)



Investigation Steps

1. Check pod events and init/sidecar readiness.
2. Verify startup sequence and readiness probes.
3. Confirm traffic interception (e.g., iptables/Envoy).
4. Check dependency initialization (e.g., DB, config, secrets).
5. Validate mesh version compatibility (control plane and sidecar).

Key Commands

```
$ kubectl get pods -n <ns> -w
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> -c istio-proxy
$ istioctl proxy-status
$ kubectl get events --sort-by=.metadata.creationTimestamp
```

What to Look For

- Sidecar readiness and startup time.
- Traffic being intercepted before dependencies are ready.
- Dependency init failures (e.g., DB connection timeout).
- Version mismatch between control plane and sidecar.

Example Evidence (Event)

```
Warning   Unhealthy   Readiness probe failed:
                      connection refused
Warning   BackOff     Back-off restarting
                      failed container
Normal    Pulling     Pulling image "istio-proxy:v1.22"
```

Root Cause

The service mesh upgrade changed the startup order and added traffic interception. The application tried to initialize its dependencies before the sidecar was ready, and a version mismatch introduced additional delays and connection failures.

Fix and Verify

- Ensure the sidecar is ready before the app starts (e.g., startupProde or init container).
- Adjust readiness/liveness probes and startup timeouts.
- Use compatible mesh versions (control plane and sidecar).
- Validate dependency availability (e.g., DB, API) at startup.
- Roll out gradually and verify pod health and logs.

Verify

```
$ kubectl rollout status deploy/<app>
$ kubectl get pods -n <ns>
$ kubectl logs <pod> -c istio-proxy
$ istioctl proxy-status
$ kubectl get events -n <ns>
```

The Autoscaler That Multiplied a CrashLoopBackOff Incident

Objective: Model how an HPA (Horizontal Pod Autoscaler) can amplify a failure by scaling out unhealthy replicas, leading to more CrashLoopBackOff pods.

Scenario

A deployment started failing at startup (CrashLoopBackOff) due to a misconfiguration. The HPA detected high CPU usage and kept scaling out, creating more failing replicas and making the incident worse.

Architecture (HPA Failure Amplification)



Investigation Steps

1. Check pod status, restart count, and failure reason.
2. Review HPA metrics and scaling events.
3. Inspect resource requests/limits (CPU/Memory).
4. Identify the root cause of the startup failure (logs, config, deps).
5. Validate a fix and control scaling during the incident.

Key Commands

```
$ kubectl get hpa
$ kubectl describe hpa <hpa-name>
$ kubectl get pods -l app=<app> -w
$ kubectl logs <pod> --previous
$ kubectl describe pod <pod-name>
```

What to Look For

- HPA scaling up repeatedly (e.g., 1 → 3 → 6 → 12).
- New pods in CrashLoopBackOff.
- High CPU/Memory metrics that trigger scaling.
- The actual startup error in pod logs.

Example Evidence (HPA)

```
NAME      REFERENCE        TARGETS   MIN  MAX  REPLICAS
app-hpa   Deployment/app   85%/70%   1    10   6
$ kubectl get pods -l app=app
NAME                STATUS             RESTARTS
app-5d7f8-abc       CrashLoopBackOff   12
app-5d7f8-def       CrashLoopBackOff   10
app-5d7f8-ghi       CrashLoopBackOff   11
```

Root Cause

A startup failure (e.g., invalid configuration, missing dependency, or wrong environment variable) caused pods to crash. The HPA saw high resource usage and scaled out, creating more failing pods and amplifying the incident.

Fix and Verify

- Fix the startup issue (config, secrets, or dependency).
- Temporarily pause or limit HPA scaling (e.g., maxReplicas).
- Use appropriate resource requests/limits.
- Add startup/readiness probes to avoid scaling on unhealthy pods.
- Verify stability and then re-enable HPA safely.

Verify

```
$ kubectl scale hpa <hpa-name> --max=3
$ kubectl rollout restart deploy/<app>
$ kubectl get pods -l app=<app>
$ kubectl get hpa <hpa-name>
```

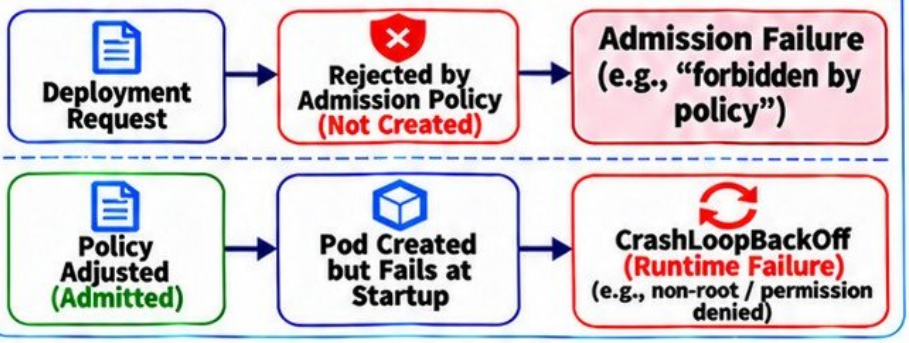
The Admission Policy Change That Broke the Next Deployment

Objective: Separate admission failures from actual CrashLoopBackOff, then reproduce a container startup failure caused by changed runtime security settings.

Scenario

A new admission policy (e.g., Pod Security Admission or OPA/Kyverno) was applied. The next deployment failed to be admitted, and after adjustments, the pod was created but crashed at startup due to stricter runtime security settings (e.g., non-root, dropped capabilities, read-only FS).

Flow: Admission vs. Runtime Failure



Investigation Steps

1. Check if the deployment was admitted or rejected.
2. Review admission policy changes (Pod Security, OPA, Kyverno).
3. If admitted, inspect pod logs and restart reason.
4. Check runtime security settings (e.g., runAsNonRoot, readOnlyRootFS, capabilities).
5. Reproduce the failure and validate a fix.

Key Commands

```
$ kubectl apply -f deploy.yaml
$ kubectl describe deploy <deploy-name>
$ kubectl get events -n <ns> --sort-by=.lastTimestamp
$ kubectl describe pod <pod-name>
$ kubectl logs <pod-name> --previous
$ kubectl get podsecuritypolicies  # or check PSA/OPA/Kyverno
```

What to Look For

- Admission failure: deployment/pod rejected (not created).
- CrashLoopBackOff: pod created, but container exits repeatedly.
- Policy changes (e.g., require non-root, drop capabilities, read-only filesystem).
- Startup logs showing permission or security errors (e.g., "permission denied", "read-only file system").

Example Evidence

```
# Admission failure (rejected)
Error: admission webhook "policy.kyverno"
denied the request: container must run as non-root

# Runtime failure (after policy adjustment)
Error: mkdir /app/data: permissionh denied
Container exited with code 1 (CrashLoopBackOff)
```

Root Cause

A new admission policy first blocked the deployment (admission failure). After updating the manifest to comply, the pod was created but still crashed because the container could not run under the new runtime security settings (e.g., non-root user, dropped capabilities, or a read-only filesystem).

Fix and Verify

- Update the manifest to meet the admission policy.
- Fix the runtime issue (e.g., proper user, writable path, required capabilities, or avoid read-only FS).
- Redeploy and confirm the pod starts successfully.

Verify

```
$ kubectl get pods -n <ns>
$ kubectl logs <pod-name>
$ kubectl describe pod <pod-name>
$ kubectl get events -n <ns>
```

The Incident Where Restarting Pods Destroyed the Evidence

Objective: Build an investigation workflow that preserves logs, events, termination reasons, metrics, and rollout metadata before pods are restarted.

Scenario

A production issue occurred, but the pods were quickly restarted (e.g., by a rollout, HPA, or manual action). The original logs and crash details were lost from the running pods, making it hard to find the root cause.

Investigation Workflow (Preserve First)

Capture Current State (Logs, Events, Status) → Preserve Previous Evidence (Previous Logs, Termination Reason) → Collect Metrics (CPU, Memory, Restarts, Trends) → Capture Rollout Metadata (Revision, History, Changes) → Analyze & Fix (Root Cause, Remediate, Prevent Recurrence)

Investigation Steps

1. Stop further restarts (if safe, e.g., pause rollout or scale to 0).
2. Collect previous logs and termination details.
3. Gather pod events and restart history.
4. Pull metrics around the incident time (CPU, memory, restarts).
5. Review rollout/replica set metadata (changes, image, config).

Key Commands

```
$ kubectl get pods -n <ns> -o wide
$ kubectl logs <pod> --previous
$ kubectl describe pod <pod>
$ kubectl get events -n <ns> --sort-by=.lastTimestamp
$ kubectl rollout history deploy/<app>
$ kubectl get rs -n <ns> -l app=<app>
```

What to Look For

- Previous container logs (before restart).
- Termination reason and exit code (e.g., OOMKilled).
- Pod events (e.g., Killing, BackOff, Created, Started).
- Metrics (CPU/Memory spike, restart count).
- Rollout metadata (revision, changed image, config).

Example Evidence (Previous Pod)

```
$ kubectl logs web-7d8f9 --previous
  Exception: connection timeout to DB
$ kubectl describe pod web-7d8f9
  Last State: Terminated
  Reason: Error  Exit Code: 1
$ kubectl get events -n prod | tail -3
  Killing container, restarting (BackOff)
```

Root Cause

Evidence was lost because the pods were restarted before the investigation captured their previous state. Without previous logs, events, and termination details, the original failure (e.g., DB connection timeout) was harder to find.

Fix and Verify

- Create an incident investigation runbook.
- Preserve evidence before any restart (logs, events, metrics, rollout data).
- Add log persistence and centralized logging (e.g., EFK/Loki/Splunk) for future incidents.

Verify

```
$ kubectl logs <pod> --previous
$ kubectl get events -n <ns> --sort-by=.lastTimestamp
$ kubectl rollout history deploy/<app>
$ kubectl describe pod <pod> | grep -E "Reason|Exit Code"
```

Final Production Challenge: Five Failures, One Kubernetes Cluster

Objective: Diagnose and resolve five interacting failures — memory pressure, bad probes, DNS disruption, secret rotation, and startup dependency failures — without indiscriminately restarting workloads.

Scenario

A single Kubernetes cluster is experiencing widespread instability. Multiple services are failing due to a combination of memory pressure, misconfigured probes, DNS issues, a rotated secret, and unmet startup dependencies.

The Five Failures (Interacting)

1. **Memory Pressure:** Nodes are under high memory usage, causing OOMKills and pod restarts.
2. **Bad Probes:** Liveness/readiness probes are too aggressive (e.g., short timeout), falsely marking pods unhealthy.
3. **DNS Disruption:** Intermittent DNS resolution failures (e.g., CoreDNS issues or exhausted resources).
4. **Secret Rotation:** A recently rotated secret (e.g., DB credentials) is not yet loated by some pods.
5. **Startup Dependency:** Critical services start before required dependencies (e.g., database, message queue) are ready.

Investigation Steps

1. Assess cluster health (nodes, events, resource usage).
2. Analyze each failure separately, then check their interactions.
3. Verify probes, DNS (CoreDNS), and secret usage in affected pods.
4. Check startup order and dependency readiness.
5. Apply targeted fixes with minimal restarts and validate stability.

Key Commands

```
$ kubectl get pods -A -o wide
$ kubectl top nodes; kubectl top pods -A
$ kubectl get events -A --sort-by=.lastTimestamp
$ kubectl describe pod <pod>
$ kubectl logs <pod> --previous
$ kubectl get secret <secret> -o yaml
$ kubectl get svc -n kube-system; kubectl logs -n kube-system
  deploy/coredns
```

What to Look For

- High memory usage, OOMKilled events, and restarts.
- Probe failures (e.g., Readiness/Liveness failed).
- DNS lookup errors (e.g., "no such host", "i/o timeout").
- Pods using stale/invalid secrets after rotation.
- Services starting before their dependencies are ready.

Example Evidence (Mixed Issues)

```
Warning   OOMKilled   pod/api-6d8f9     (memory limit)
Warning   Unhealthy   Readiness probe failed:
                      HTTP probe timeout
Warning   FailedDNS   lookup db.service.svc.cluster.local:
                      no such host
Normal    Pulled      Using new secret version
Warning   BackOff     Waiting for database at startup
```

Root Cause (Combined)

Multiple interacting issues: memory pressure caused restarts, misconfigured probes amplified the failures, DNS instability broke service resolution, the secret rotation caused authentication failures, and services started before their dependencies were ready.

Fix and Verify

- Relieve memory pressure (increase limits or reduce usage).
- Fix probes (use realistic initialDelay, timeout, and thresholds).
- Restore DNS health (check CoreDNS, resources, and config).
- Roll out the rotated secret correctly and verify usage.
- Enforce startup ordering (initContainers or readiness checks).

Verify

```
$ kubectl get pods -A
$ kubectl top pods -A
$ kubectl get events -A
$ kubectl logs <pod> --tail=100
$ kubectl rollout status deploy/<app>
```